QA LEADERSHIP ACADEMY

# QA Capability Heatmap

A one-page grid of team-versus-capability that exposes single points of failure, over-reliance on individuals and where to hire or train next.

## Purpose

A capability heatmap maps who can do what, at what depth, across your team. Where the maturity assessment tells you how good the organisation’s practices are, the heatmap tells you how resilient your people are: where knowledge is concentrated in one person, where a whole capability is missing, and where you are one resignation away from a crisis. It turns an anxious feeling ("we’d be in trouble if Dan left") into a visible, discussable fact.

## When to use it

- When planning hiring or training — to target the real gaps rather than the loudest requests.
- For succession and bus-factor planning — to find capabilities that rest on a single person.
- Before taking on new scope (a new product surface, a performance or security remit) to check whether you have the skills to cover it.
- In performance and development conversations, to agree concrete growth targets with each engineer.

## Instructions

List your capabilities down the rows and your people across the columns. Rate each person against each capability using the four-point scale below. Keep the scale simple — the value is in the pattern, not in precise scoring. Fill it in with input from the people themselves; self-assessment plus your calibration is far more accurate than either alone. Then read the grid by rows (capability resilience) and by columns (individual breadth and load).

| Rating | Meaning |
|---|---|
| — (None) | No working knowledge. Could not do this unaided. |
| L — Learning | Can contribute with guidance and review. Growing into it. |
| P — Proficient | Works independently and reliably. A dependable pair of hands. |
| E — Expert | Sets the standard; can coach others and handle the hard cases. |

## Worked example

Here is the heatmap for Northstar Digital’s six QA engineers: Sofia (senior, exploratory strength), Dan (automation), Aisha and Ben (strong manual/exploratory), Grace (manual/exploratory), and Ravi (recent hire, still ramping).

| Capability | Sofia | Dan | Aisha | Ben | Grace | Ravi |
|---|---|---|---|---|---|---|
| Exploratory testing | E | P | P | P | P | L |
| Test design techniques | E | P | P | P | L | L |
| API testing | P | P | L | L | — | — |
| UI automation (Selenium) | L | E | — | L | — | — |
| CI/CD & pipelines | — | P | — | — | — | — |
| Test data management | L | L | — | — | — | — |
| Performance testing | — | L | — | — | — | — |
| Security testing | — | — | — | — | — | — |
| Accessibility testing | L | — | L | — | — | — |
| Payments / billing domain | P | L | P | — | — | — |
| Mentoring / coaching | E | — | L | L | — | — |

The pattern is immediately legible. Exploratory testing is deep and resilient — a genuine strength. But UI automation and CI/CD rest almost entirely on Dan: if he leaves, that capability collapses, which matters because he is already close to burning out. Security testing is a complete blank. Performance and test-data management are thin single-person capabilities. Payments-domain knowledge sits with only three people.

## Blank reusable version

| Capability | Person 1 | Person 2 | Person 3 | Person 4 | Person 5 |
|---|---|---|---|---|---|
| Exploratory testing | | | | | |
| Test design techniques | | | | | |
| API testing | | | | | |
| UI automation | | | | | |
| CI/CD & pipelines | | | | | |
| Test data management | | | | | |
| Performance testing | | | | | |
| Security testing | | | | | |
| Accessibility testing | | | | | |
| Domain knowledge (name it) | | | | | |
| Mentoring / coaching | | | | | |

Use — / L / P / E in each cell. Add or remove capability rows to match your context, and add columns for each team member.

## Common mistakes

**COMMON MISTAKES**

Using too many rating levels — a ten-point scale gives false precision and hides the pattern. Rating people without their input, which produces a grid they will not trust or own. Conflating job title with capability (a "senior" is not automatically Expert at everything). Treating the heatmap as a stack-ranking or a performance verdict — it is a map of capability distribution, not a league table, and framing it as the latter will poison the team's honesty next time. Finally, filling it in once and never revisiting it as people grow and leave.

## How to interpret the results

- A row with a single E or P and everything else blank — a single point of failure. Prioritise cross-training or documentation, especially if the capability guards real risk. (At Northstar: UI automation and CI/CD on Dan.)
- A completely blank row — a missing capability. Decide deliberately whether to hire, train, borrow from another team, or accept the gap. (At Northstar: security testing.)
- A column that is thin across the board — someone early in their development. Pair them with an expert and set explicit growth targets. (At Northstar: Ravi.)
- A column that is E or P everywhere — a heavily-relied-upon individual. Check their load and retention risk; over-reliance is fragile even when it feels comfortable.
- A dense, well-distributed grid — a resilient team. Your investment can shift from resilience towards depth and new capabilities.

**INSIDE STLC PRO TIP**

Overlay the heatmap on your risk picture before you act. A single point of failure in a low-risk capability can wait; a single point of failure guarding your highest-risk flow is an urgent business problem you can now name precisely to your manager — which is exactly how a training or hiring request gets approved.